Finding P0–P4 in VFE quality-control traces with a learned peak finder

Santanu

informal lab discussion — 10 November 2026

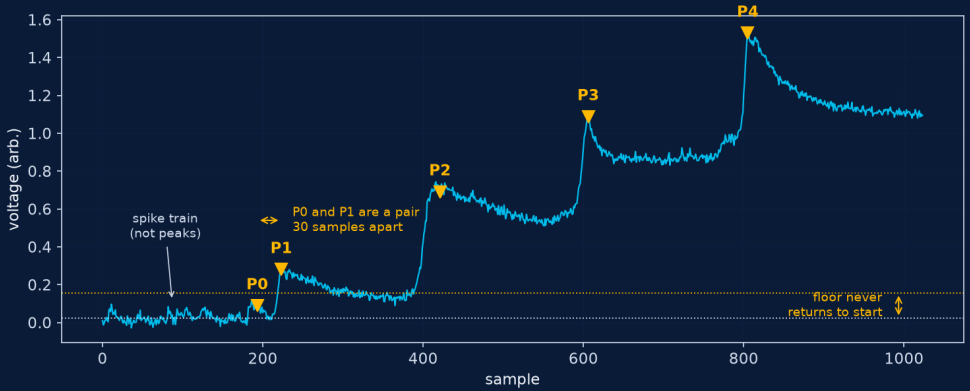
status: toy study only, no board data yet

Where this comes from

- VFE board QC gives a **voltage trace** per board. Five landmarks have to be read off it: **P0, P1, P2, P3, P4**.
- Today a **hard-coded finder** does this — fixed thresholds and windows.
- It **misidentifies peaks** on some boards. Every miss needs a human to look at the trace.
- Idea: learn the peak positions from labelled traces instead of hand-tuning thresholds, and get a **confidence per peak** so the doubtful boards flag themselves.

Everything shown here is measured on simulated traces built from the description of the real waveform — see the next slide.

What the trace looks like



One simulated trace. Gold markers are the five true landmark positions; P0 and P1 sit close together, 10–60 samples apart. Amplitude is in arbitrary units, shape only — the absolute scale is not modelled.

Why thresholds struggle

- **P0 hides in the spike train.** It is the *last and smallest* of many little spikes — nothing about its height singles it out.
- **P0 then sits right in front of P1**, a few tens of samples away and far smaller. A finder that keys on height takes the pair as one structure.
- **The floor never resets.** After each peak the baseline settles **higher** than before, so one global threshold cannot serve all five.
- **There is structure that is not a peak.** Wiggles ride on the raised floors and must be ignored.
- **The shape varies.** Rise and fall widths are drawn independently: a peak can rise slowly and fall fast, or the other way round.
- Positions are not fixed — the five landmarks move trace to trace.

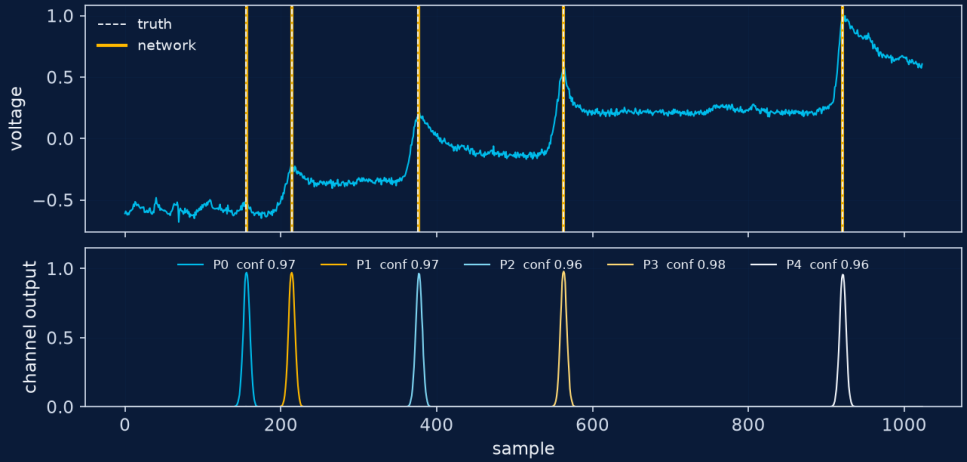
The approach

A fully convolutional network reads the whole trace and emits **five output channels**, one per landmark.

- Each channel is trained to peak at its own landmark — a narrow bump, width **4 samples**, centred on the true position.
- Position = where the channel is largest. **Confidence = how large it gets** — that is the per-peak quality flag.
- Channels are separate, so **P0 cannot be confused with P4**: each one only has to find its own landmark.
- Each landmark is reported with **the apex beside it** — the nearest local maximum and its height, in the board's own units. If that apex is clearly higher than the trace at the reported position and the channel is unsure, the position moves onto it.
- Trained on **8000 simulated traces**, tested on 1000 held out. Input is median-subtracted and scaled per trace, so absolute gain does not matter.

Alternative considered: regress five numbers directly. Rejected — it gives no per-peak confidence, which is the part QC actually needs.

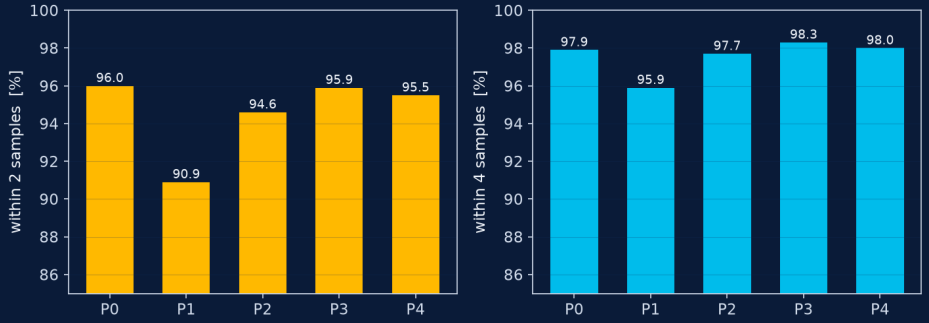
One test trace



Top: a held-out trace, true positions dashed, network output solid. Bottom: the five channel outputs, with the confidence each one reaches. Amplitude normalised per trace.

How well it does — on simulation

All five within 2 samples on **76.8%** of traces, within 4 on **89.0%**. Badly wrong ($>$ 15 samples) on **2.0%**.

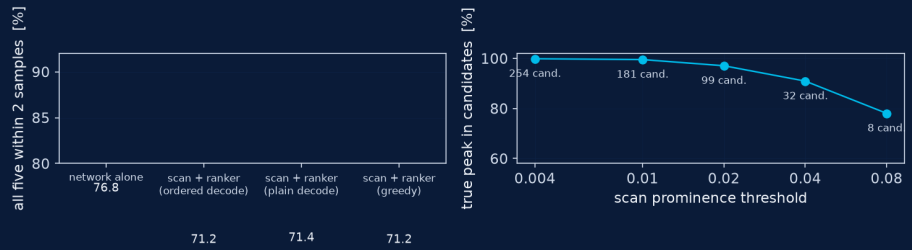


1000 held-out simulated traces. Fraction of traces where each landmark lands within 2 (left) and 4 (right) samples of the visible apex. Suppressed zero on both.

Labels sit on the visible apex. Misses are mostly 3 samples — just over the line, no early/late bias. On P3 and P4 most of them land on a flat top within 1% of the apex height.

A hybrid I tried — and dropped

Two-stage idea: scan the trace for local maxima, then let a classifier decide which candidate is which landmark.



Left: fraction of traces with all five landmarks within 2 samples. Right: how often the true landmark is among the scanned candidates, against the scan threshold; the label gives candidates per trace.

With labels on the visible apex the scan finds the true peak 99.9% of the time, so recall is not the issue — the re-ranker is simply a worse judge than the network it is reading.

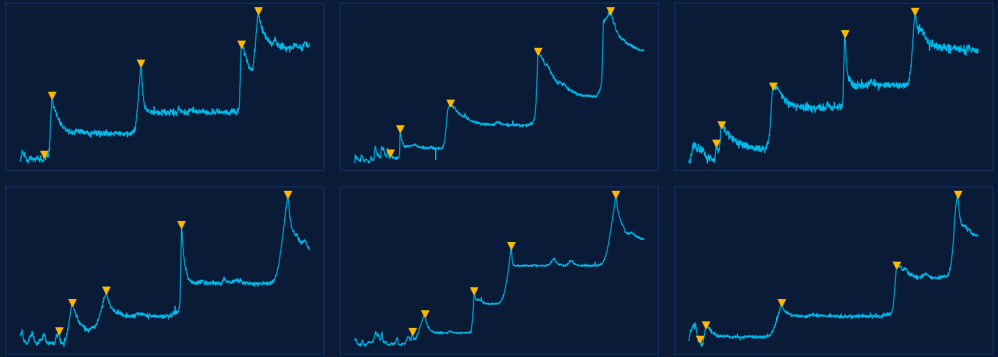
What I need next

- **Real traces.** Everything above is simulation written from a verbal description — the numbers are a sanity check, not a result.
- **Labels on the hard cases** — especially traces the present finder gets wrong. A few hundred is enough to fine-tune.
- **One detail to check:** a peak is its **visible apex** — but does the current finder take the apex of the **raw** trace or of a smoothed one? Against the raw-trace apex, snapping to it gains **20 points**.
- Then: refit the simulation to measured traces, fine-tune, and report this same table on real held-out boards.

Backup

Backup: more simulated traces

six draws from the generator, same settings



Six independent draws, identical generator settings. Gold markers are the true landmarks. Each panel is auto-scaled; shape only, no common vertical scale.

Backup: how a trace is built

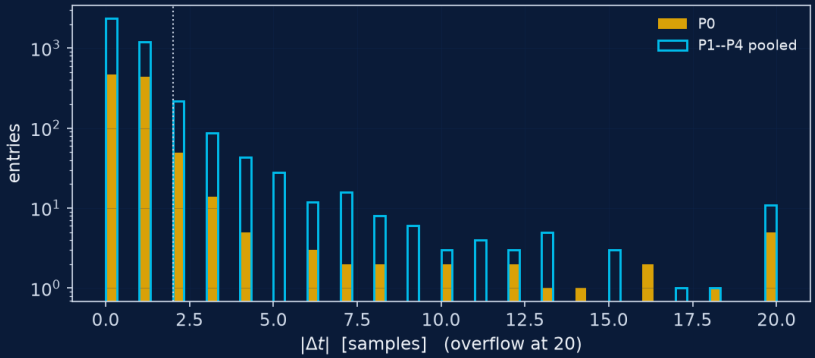
Every number below is a guess from the verbal description. **These are what real traces will replace.**

ingredient	range used
trace length	1024 samples
P1–P4 live in	samples 143–922, evenly spread
P0 apex ahead of P1 apex by	10–60 samples
spikes before P0	8–20, height 0.01–0.06
P0 height	0.04–0.09 (top of the spike family)
P1–P4 heights	0.25–1.00, increasing within a trace
leading-edge width	1.5–14 samples
trailing decay	2.5–30 samples, drawn independently
per-trace shape tilt	0.5–2.0 (slow rise $\leftrightarrow$ fast rise)
fraction of a peak left as floor	0.25–0.55
floor step width	3–10 samples
decoy bumps per floor	2–6, at 4–18% of the local step
noise	0.004–0.015, plus 0.2% single-sample glitches
label	visible apex; 42% of draws redrawn
landmark spacing jitter	50% of the mean spacing

Backup: network and training

- **U-shaped 1D convolutional network** — three downsampling levels, three up, skip connections, kernel width 9. **462k parameters**.
- Output: five channels at full trace length. Target is a Gaussian of width **4 samples** on the true position; loss is per-sample binary cross-entropy against it.
- Adam, learning rate 10^{-3} with cosine decay, batch 32, **25 epochs**. **125 s** on one GPU.
- Input per trace: subtract the median, divide by the largest absolute value. **Any inference path must repeat this exactly** — it is part of the model.
- Labels and errors use the **visible apex** of the noiseless waveform. Draws where a landmark has no visible apex are redrawn.

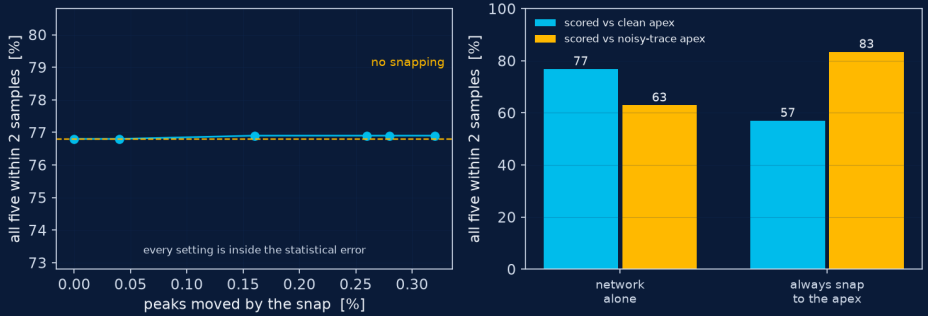
Backup: where the failures are



Held-out simulated traces: P0 (1000 entries, filled) against P1–P4 pooled (4000 entries, outline). Log vertical scale, dotted line at the 2-sample criterion, everything beyond 20 piled in the last bin.

P0 misses on 40 traces, 8 badly. About half land earlier, in the spike train; none land on P1.

Backup: snapping to the apex



Left: moving each reported position onto the nearby apex of the noisy trace, against how many positions get moved. Right: the same two methods scored against the clean apex and against the noisy-trace apex. 1000 held-out simulated traces.

Scored against the clean apex, snapping does nothing. Scored against the apex of the noisy trace, always snapping gains **20 points** — so it matters which of the two the present finder reports.

Backup: the hybrid in numbers

Same 1000 held-out traces; the re-ranker was trained on 2000 separate ones.

method	all five within 2	badly wrong
network alone	**76.8%**	**2.0%**
scan + ranker, ordered decode	71.2%	2.1%
scan + ranker, plain decode	71.4%	2.1%
scan + ranker, greedy	71.2%	2.4%

- Ordering constraints are **not** the problem: ordered and plain decode agree to a fifth of a percent, and the network's own answer breaks the ordering on only **1 of 1000** traces.
- After adding the network's own maxima to the candidate list, the true landmark is in the list on **1000 of 1000** traces — so recall is not the problem either.
- Conclusion: the second-stage ranker is a **worse judge than the network output it reads**. Kept as a switch, not the default.